

Relatório de Auditoria de Segurança

SIC-IA — Sistema Inteligente de Cardápio

Data: 16 de Setembro de 2026

Escopo: Backend (Railway) + Frontend (Vercel) + Cart flow

Nota Metodológica: Cada categoria de auditoria foi mapeada para a stack detectada (Node.js + Express 5.2.1 + Prisma 6.5.0 + PostgreSQL/Supabase + JWT + Vanilla JS). O mecanismo de isolamento de tenant identificado é o middleware `resolveEmpresa` + filtro manual por `empresaId` em queries Prisma. Não foram encontradas políticas de RLS no Supabase. Auditoria manual de todo o código-fonte, repositório a repositório, linha a linha.

2 Críticas

2 Altas

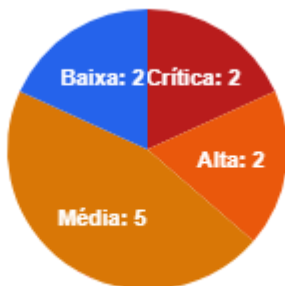
5 Médias

2 Baixas

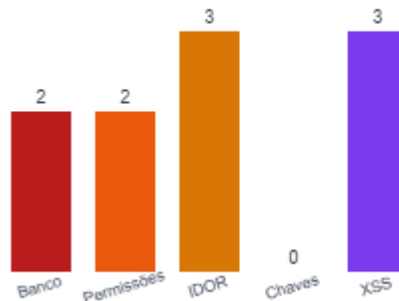
6 Pontos Fortes

1. Resumo Executivo

Achados por Severidade



Achados por Categoria



Esta auditoria identificou **11 achados de segurança** no projeto SIC-IA, incluindo **2 vulnerabilidades críticas** que permitem acesso não autenticado a dados pessoais de clientes (IDOR) e destruição de dados cross-tenant. A maioria dos controles de segurança está bem implementada (autenticação JWT, rate limiting, CSP, escapeHtml), mas falhas pontuais criam vetores de exploração significativos.

Pontos Fortes

SF-1: Todas as rotas admin/superadmin validam roles no backend com `authenticate + authorize('superadmin')`. O gateway de permissão no navegador é cosmético — o backend é o guard real.

SF-2: Função `escapeHtml()` completa (&, <, >, ", ') é usada consistentemente em `admin.js`, `cart.js`, `menu.js`, `filiais.js` para dados de usuário em `innerHTML`.

SF-3: Proxy routes (mapbox, geoapify, graphhopper) têm allowlist de paths + rate limit + serviços hardcoded. SSRF mitigado.

SF-4: Webhooks (ASAAS, iFood, Keeta, Food99) verificam tokens em headers. Autenticação adequada.

SF-5: Nenhuma API key ou secret hardcoded no código-fonte. Todos os segredos via `process.env`.

SF-6: JWT_SECRET não tem default inseguro — `env.js:5` retorna `undefined` se não setado. Falha visível no startup.

2. Categoria 1: Banco sem Tranca (Isolamento de Tenant)

Mecanismo de isolamento: `middleware resolveEmpresa` + filtro manual `empresaId` em queries Prisma. Sem RLS no Supabase. Isolamento é 100% application-level.

CRÍTICA C1-1: Destruição cross-tenant de webhooks

Arquivo: `backend/src/repositories/empresaRepository.js:36`

```
prisma.processedWebhook.deleteMany({ where: {} })
```

Por que é explorável: Ao deletar uma empresa, TODOS os webhooks processados de TODAS as empresas são deletados. Um admin de empresa A pode causar perda de dados em empresa B ao deletar sua própria empresa.

Condição: Admin autenticado executa exclusão de empresa.

ALTA C1-2: Sem RLS no Supabase

Arquivo: Nenhum arquivo SQL/migration com políticas RLS encontrado.

Por que é explorável: Qualquer falha no middleware resolveEmpresa ou em uma rota que esqueça o filtro resulta em vazamento de dados cross-tenant direto no banco.

Condição: Falha em qualquer rota que não filtre por empresald.

3. Categoria 2: Permissão Definida no Navegador

Backend valida roles em TODAS as rotas sensíveis. O frontend tem gates cosméticos.

MÉDIA C2-1: superadmin.html sem validação de role no frontend

Arquivo: superadmin.html:416

```
if(!authGuard()) throw new Error('Redirect');
```

Por que é explorável: authGuard() só verifica autenticação, não role. Qualquer usuário autenticado (admin, user, entregador) pode acessar a página superadmin no frontend.

Impacto real: BAIXO — backend valida authorize('superadmin') em todas as rotas.

MÉDIA C2-2: admin.js sem validação de role no frontend

Arquivo: js/admin.js:17-28

```
// Apenas redireciona admin para superadmin, não valida role
if (!isLocal && (sub === 'admin' || sub === 'www')) {
  window.location.href = '/superadmin.html';
}
```

Por que é explorável: Qualquer usuário autenticado pode acessar admin.html.

Impacto real: BAIXO — backend valida roles em todas as rotas admin.

4. Categoria 3: IDOR

CRÍTICA C3-1: buscarPedido sem autenticação — IDOR completo

Arquivo: backend/src/controllers/publicController.js:462-468

```
exports.buscarPedido = asyncHandler(async (req, res) => {
  const empId = requireTenant(req, res);
  if (!empId) return;
  const pedido = await sql.buscarPedido(req.params.id, empId);
  if (!pedido) return res.status(404).json({ error: 'Pedido não encontrado' });
  res.json(pedido);
});
```

Por que é explorável: Rota pública sem authenticatePublic. Retorna dados completos do pedido: clienteNome, clienteWhatsapp, clienteEndereco, clienteNumero, clienteBairro, clienteCep, clienteReferencia, itens, preços.

IDs sequenciais: {empresaId}-{001, 002, 003...} — enumerável em segundos.

Exemplo: GET /api/public/pedidos/7-001?slug=salgadoscosta retorna dados do cliente.

ALTA C3-2: IDs de pedido sequenciais e previsíveis

Arquivo: backend/src/repositories/sqlRepository.js:177-184

```
return `${empresaId}-${String(counter.lastValue).padStart(3, '0')}`;
```

Por que é explorável: IDs incrementais (7-001, 7-002...) permitem enumeração trivial. Combinado com C3-1, qualquer pessoa pode acessar todos os pedidos.

BAIXA C3-3: validarCupom sem autenticação

Arquivo: backend/src/controllers/publicController.js:470-477

```
exports.validarCupom = asyncHandler(async (req, res) => {
```

Por que é explorável: Permite validação de cupons sem autenticação. Risco baixo — apenas revela existência e desconto.

5. Categoria 4: Chaves Expostas

PONTO FORTE C4-1: Nenhuma chave hardcoded encontrada

Todas as API keys, JWT secrets e tokens são carregados de process.env via env.js.

PONTO FORTE C4-2: JWT_SECRET sem default inseguro

Arquivo: backend/src/config/env.js:5

```
jwtSecret: process.env.JWT_SECRET,
```

Retorna undefined se não setado. jwt.sign() falha visivelmente.

6. Categoria 5: Inputs sem Tratamento (XSS)

MÉDIA C5-1: innerHTML sem escapeHtml em admin.js (driver modal)

Arquivo: js/admin.js:426,431,432

```
pre>overlay.innerHTML = ` ...
```

Atribuir entregador para o pedido de \${pedidoNome}

```
... ${d.nome} 📞 ${d.whatsapp || '-'}`
```

Por que é explorável: Se d.nome ou d.whatsapp contiver HTML malicioso, será renderizado.

Condição: Usuário malicioso cria entregador com nome contendo <script>.

MÉDIA C5-2: innerHTML sem escapeHtml em admin.js (edit modal)

Arquivo: js/admin.js:569

```

'
Cliente: ' + (p.clienteNome || p.cliente?.nome || '-') + '
'

```

Por que é explorável: Nome do cliente inserido sem escape no modal de edição.

MÉDIA C5-3: CSP com unsafe-inline em scriptSrc

Arquivo: backend/src/app.js:75-76

```

scriptSrc: ["'self'", "'unsafe-inline'", ...],
scriptSrcAttr: ["'unsafe-inline'"],

```

Por que é explorável: Permite XSS via inline scripts e event handlers.

7. Recomendações Priorizadas

Prioridade	Achado	Ação	Arquivo
P1	C3-1: IDOR buscarPedido	Adicionar authenticatePublic + filtro por clienteWhatsapp	publicController.js:462
P1	C1-1: deleteMany cross-tenant	Filtrar processedWebhook por empresald	empresaRepository.js:36
P1	C3-2: IDs sequenciais	Usar UUID ou hash opaco para IDs de pedido	sqlRepository.js:177
P2	C1-2: Sem RLS	Configurar RLS no Supabase como defesa em profundidade	Supabase Dashboard
P2	C5-1/C5-2: XSS admin.js	Aplicar escapeHtml em pedidoNome, d.nome, d.whatsapp	admin.js:426,569
P2	C5-3: CSP unsafe-inline	Remover unsafe-inline, usar nonces	app.js:75
P3	C2-1/C2-2: Frontend role gates	Adicionar verificação de role no authGuard()	utils.js:63
P3	C3-3: validarCupom sem auth	Considerar adicionar authenticatePublic	publicController.js:470

8. Issues para o GitHub

--- ISSUE 1 ---

[Segurança] IDOR crítico — buscarPedido expõe dados de clientes sem autenticação**Labels:** security, critical**Problema:** A rota `GET /api/public/pedidos/:id` não possui autenticação. Qualquer pessoa pode acessar dados pessoais de qualquer pedido (nome, telefone, endereço, CEP) bastando incrementar o ID sequencial.**Evidência:** `backend/src/controllers/publicController.js:462-468`

```
exports.buscarPedido = asyncHandler(async (req, res) => {
  const empId = requireTenant(req, res);
  if (!empId) return;
  const pedido = await sql.buscarPedido(req.params.id, empId);
  if (!pedido) return res.status(404).json({ error: 'Pedido não encontrado' });
  res.json(pedido); // ← Sem autenticação!
});
```

Impacto: Vazamento de dados pessoais de clientes (LGPD). Qualquer pessoa pode enumerar pedidos.**Sugestão de correção:** Adicionar `authenticatePublic` e filtrar por `clienteWhatsapp`.**Critérios de aceite:**

- [] Rota requer Bearer token válido
- [] Usuário só acessa seus próprios pedidos
- [] Teste E2E verifica que pedido de outro usuário retorna 404

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] Destruição cross-tenant — deleteMany sem filtro em processedWebhook**Labels:** security, critical**Problema:** Ao deletar uma empresa, `processedWebhook.deleteMany({ where: {} })` deleta TODOS os webhooks processados de TODAS as empresas, causando perda de dados cross-tenant.**Evidência:** `backend/src/repositories/empresaRepository.js:36`

```
prisma.processedWebhook.deleteMany({ where: {} }) // ← Sem filtro de empresaId!
```

Impacto: Perda de dados de webhook para todas as empresas. Webhooks podem ser reprocessados.**Sugestão de correção:** Adicionar filtro `{ where: { pedido: { empresaId } } }` ou remover esta linha se `processedWebhook` não tem relação com empresa.**Critérios de aceite:**

- [] `deleteMany` filtra por `empresald`
- [] Teste verifica que deletion não afeta outras empresas

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] IDs de pedido sequenciais permitem enumeração trivial**Labels:** security, high**Problema:** IDs de pedido são incrementais (7-001 , 7-002 ...), facilitando enumeração.**Evidência:** backend/src/repositories/sqlRepository.js:177-184

```
return `${empresaId}-${String(counter.lastValue).padStart(3, '0')}`;
```

Impacto: Combinado com C3-1, qualquer pessoa pode acessar todos os pedidos.**Sugestão de correção:** Usar UUID ou ID opaco (cuid() , uuid()).**Critérios de aceite:**

- [] IDs de pedido são UUIDs ou opacos
- [] Counter interno mantém sequência para display
- [] Teste verifica que IDs antigos não são enumeráveis

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Segurança] XSS em admin.js — innerHTML sem escapeHtml em modais**Labels:** security, medium**Problema:** Nomes de entregadores e clientes são inseridos em innerHTML sem escapeHtml() em admin.js.**Evidência:**

- js/admin.js:426 — `\${pedidoNome}` sem escape
- js/admin.js:431 — `\${d.nome}` sem escape
- js/admin.js:432 — `\${d.whatsapp}` sem escape
- js/admin.js:569 — p.clienteNome sem escape

Impacto: Stored XSS se usuário malicioso criar nome com HTML/JS.**Sugestão de correção:** Aplicar escapeHtml() em todas as interpolações.**Critérios de aceite:**

- [] Todos os dados de usuário em innerHTML usam escapeHtml
- [] Teste verifica que nome com <script> é renderizado como texto

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

[Segurança] CSP com unsafe-inline permite XSS**Labels:** security, medium**Problema:** Content-Security-Policy permite 'unsafe-inline' em scriptSrc e scriptSrcAttr.**Evidência:** backend/src/app.js:75-76

```
scriptSrc: ["'self'", "'unsafe-inline'", ...],
scriptSrcAttr: ["'unsafe-inline'"],
```

Impacto: Permite injeção de scripts inline e event handlers.**Sugestão de correção:** Remover unsafe-inline, usar nonces para scripts inline.**Critérios de aceite:**

- [] scriptSrc não contém unsafe-inline
- [] scriptSrcAttr não contém unsafe-inline
- [] Scripts inline funcionam com nonces

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

[Segurança] Frontend role gates são cosméticos — adicionar verificação no authGuard**Labels:** security, low**Problema:** authGuard() só verifica autenticação, não role. Qualquer usuário autenticado pode acessar superadmin.html e admin.html no frontend.**Evidência:** js/utils.js:63-80 — authGuard não verifica role.**Impacto:** BAIXO — backend valida roles. Mas UI confunde usuários e expõe funcionalidades desnecessárias.**Sugestão de correção:** Adicionar parâmetro de role em authGuard(requiredRole) .**Critérios de aceite:**

- [] authGuard aceita role opcional
- [] superadmin.html chama authGuard('superadmin')
- [] admin.html chama authGuard('admin')

--- FIM ISSUE 6 ---